BOTCHAIN ECOSYSTEM

GPA Vault

Decentralised Academic Credential Protocol

Technical Whitepaper · Version 1.0 · October 2026

A privacy-preserving, on-chain system for computing, anchoring, and verifying academic GPA records — with student identity permanently linked to a Botchain wallet.

Authors	GPA Vault Core Team
Chain	Botchain (Chain ID 677)
Contract	GPAVault.sol — Solidity ^0.8.20
License	MIT
Status	Draft for Public Review

Table of Contents

1 Abstract

2 Introduction

2.1 Motivation

2.2 Design Goals

3 System Architecture

3.1 Component Overview

3.2 Data Flow

4 Smart Contract Specification

4.1 GPAVault.sol

4.2 Function Reference

4.3 Events

5 Registration Protocol

5.1 Student Identity Onboarding

5.2 Profile Data Model

6 GPA Computation Engine

6.1 Grading Scales

6.2 Degree Classification

7 Privacy & Hashing Model

7.1 SHA-256 Payload Schema

7.2 What Is and Is Not Stored On-Chain

8 Verification Protocol

9 Botchain Integration

9.1 Network Parameters

9.2 Gas & Cost Model

10 Security Considerations

11 Roadmap

12 Conclusion

13 References

1 Abstract

GPA Vault is an open-source, privacy-first academic credential protocol built on Botchain. Students register their identity on-chain once — binding their name, matric number, and institution to a wallet address — then periodically anchor a SHA-256 hash of their locally-computed GPA payload to the blockchain. The full transcript never leaves the student's device. Third parties verify authenticity by comparing a submitted hash against the on-chain record, requiring no trusted intermediary and no centralised database. The protocol supports both 4.0 and 5.0 grading scales, multi-semester tracking, and the West African university degree classification system.

2 Introduction

Academic credentials remain one of the most friction-heavy documents in the professional world. Transcripts are printed on paper, couriered across continents, verified by phone calls to registrar offices, and susceptible to forgery. Centralised digital solutions shift the trust problem to a single point of failure — the issuing institution's portal or a third-party credential platform.

GPA Vault re-frames the question: instead of moving the credential, move only its fingerprint. A deterministic SHA-256 hash of the student's full result payload — computed locally, anchored on an immutable public ledger — gives any verifier the ability to confirm authenticity without seeing the underlying data.

2.1 Motivation

- Nigerian and West African universities issue paper transcripts that take weeks to verify internationally.
- Employers and graduate schools have no cryptographic way to confirm a result slip has not been altered.
- Students have no self-sovereign control over their academic records.
- Existing blockchain credential schemes require institutions to be the issuers, excluding millions of self-reporting students.

2.2 Design Goals

Privacy-first	Raw transcript data never touches the blockchain or any server.
Self-sovereign	Students control registration and record storage without institutional permission.
Trustless verification	Any party can verify a (address, hash) pair from a public RPC node.
Simplicity	The contract is under 120 lines of Solidity. No oracles, no upgradeable proxies.

Scale-agnostic	Supports 4.0 (US) and 5.0 (West African / Nigerian) grading scales.

3 System Architecture

3.1 Component Overview

GPA Vault comprises three loosely-coupled layers: a client-side computation engine, a registration and record-storage smart contract, and a verification interface. No backend server exists. All state transitions happen on-chain; all sensitive data stays local.

Layer	Component	Technology	Role
Client	GPA Calculator	HTML / JS (Vanilla)	Compute GPA, build payload, generate hash
Client	Registration UI	HTML / JS + ethers.js	Connect wallet, call register(), call storeRecord()
On-Chain	GPAVault Contract	Solidity ^0.8.20	Store identity + hashes, expose view functions
Network	Botchain	EVM-compatible L1	Settle transactions (Chain ID 677, BOT token)
Verifier	Verification UI	HTML / JS + ethers.js	Call verifyRecord() and display result

Table 1 — System component overview

3.2 Data Flow

1. Student connects MetaMask wallet to the GPA Vault web app.
2. Student fills name, matric number, institution and submits the registration form.
3. App calls GPAVault.register(name, matricNo, institution) — one on-chain write.
4. Student enters course codes, titles, credit units, and scores into the calculator.
5. App builds a JSON payload locally and computes SHA-256(payload) in the browser.
6. Student calls GPAVault.storeRecord(hash, label) — second on-chain write.
7. Verifier queries GPAVault.verifyRecord(studentAddress, hash) — read-only, free.

4 Smart Contract Specification

4.1 GPAVault.sol

The contract is intentionally minimal — no upgradeable proxy, no admin key, no fee. It stores two mappings: student identity (address → Student struct) and records (address → Record[]). Both are append-only by design; deletion is not supported to maintain an immutable audit trail.

Storage layout

```
mapping(address => Student) public students
mapping(address => Record[]) private _records

struct Student { string name; string matricNo; string institution; uint256 registeredAt; bool exists; }
struct Record { bytes32 dataHash; uint256 timestamp; string label; }
```

4.2 Function Reference

Function	Visibility	Description
`register(name, matricNo, institution)`	external	Write student identity on-chain. Re-callable to update profile fields. Emits Registered.
`storeRecord(dataHash, label)`	external	Append a SHA-256 record. Requires prior registration. Emits RecordStored.
`getStudent(address)`	view	Return (name, matricNo, institution, registeredAt) for a given address.
`getRecordCount(address)`	view	Return total number of stored records for the address.
`getRecord(address, index)`	view	Return (dataHash, timestamp, label) for a specific record index.
`getLatestRecord(address)`	view	Convenience helper — returns the most recent record.
`verifyRecord(address, dataHash)`	view	Return (found bool, timestamp uint256). Iterates records in reverse.
`isRegistered(address)`	view	Return true if the address has called register() at least once.

Table 2 — GPAVault function reference

4.3 Events

Function	Visibility	Description
`Registered(address indexed, name, matricNo, institution)`	event	Emitted on every successful register() call. Indexable by student address.
`RecordStored(address indexed, bytes32 indexed, label, timestamp)`	event	Emitted on every storeRecord() call. Both address and hash are indexed for efficient off-chain querying.

Table 3 — Contract events

5 Registration Protocol

5.1 Student Identity Onboarding

Registration is a one-time, self-sovereign operation. The student supplies three strings (full name, matric/student number, institution name) and signs the transaction with their private key. No email, no password, no KYC. The wallet address becomes the permanent student identifier. The protocol deliberately excludes date-of-birth or other PII from on-chain storage.

The app's registration modal first calls isRegistered(address) as a read (free). If the address is already registered, the stored profile is fetched and displayed, allowing returning users to proceed directly to their dashboard without a second write transaction.

Returning user optimisation: The front-end caches registration state in localStorage. On subsequent visits, the landing page is bypassed entirely and the app opens directly to the dashboard. The cache is validated against the contract whenever a contract address is configured.

5.2 Profile Data Model

Field	Type	On-Chain?	Notes
name	string	Yes	Full legal name as the student chooses to present it.
matricNo	string	Yes	Student/matric number — institution-assigned.
institution	string	Yes	University or college name.
registeredAt	uint256	Yes	Unix timestamp of the register() call.
wallet address	address	Yes (implicit key)	msg.sender at time of registration.
Full transcript	JSON	No	Stays on device. Only the SHA-256 hash is stored.
CGPA	float	No	Computed locally; included in the hashed payload, not stored raw.

Table 4 — Profile data model

6 GPA Computation Engine

All computation happens in the browser in plain JavaScript. No server receives scores. The engine supports concurrent semester tracking, automatic grade assignment, and quality-point accumulation.

6.1 Grading Scales

The engine supports two scales selectable per session. Both use the same score thresholds; only the grade-point mapping differs:

Score Range	Grade	Points (5.0 scale)	Points (4.0 scale)
70 – 100	A	5.0	4.0
60 – 69	B	4.0	3.0
50 – 59	C	3.0	2.0
45 – 49	D	2.0	1.0
40 – 44	E	1.0	F (0.0)
0 – 39	F	0.0	0.0

Table 5 — Grading scale comparison

6.2 Degree Classification

CGPA Range (5.0)	CGPA Range (4.0)	Classification
4.50 – 5.00	3.60 – 4.00	First Class Honours
3.50 – 4.49	3.20 – 3.59	2nd Class Honours (Upper Division)
2.40 – 3.49	2.40 – 3.19	2nd Class Honours (Lower Division)
1.50 – 2.39	1.60 – 2.39	Third Class Honours
1.00 – 1.49	1.00 – 1.59	Pass
0.00 – 0.99	0.00 – 0.99	Fail

Table 6 — Degree classification thresholds

---

7 Privacy & Hashing Model

The core privacy guarantee is simple: the contract stores 32 bytes. Nothing else. The full result data — course codes, titles, scores, computed grades — exists only in the student's local browser storage. When a student wants to prove their CGPA to a verifier, they share the raw JSON payload. The verifier hashes it with SHA-256 and compares it to what is on-chain.

7.1 SHA-256 Payload Schema

The JSON payload is serialised with deterministic key ordering and 2-space indentation before hashing. The schema is:

```
{"student": "0x...", "name": "Jeremiah Adedayo", "matricNo": "CSC/2023/001",
"institution": "LAUTECH",
  "scale": "5", "cgpa": 4.7231, "units": 120, "classification": "First Class
Honours",
```

```
  "semesters": [ { "name": "100L Harmattan", "gpa": 4.80, "units": 30,
    "courses": [ { "code": "CSC101", "title": "Intro to CS", "units": 3, "score": 78,
"grade": "A", "points": 5 } ] } ],
  "generated": "2026-10-30T12:00:00.000Z"
}
```


Hash: `SHA-256(JSON.stringify(payload, null, 2))` — computed via the Web Crypto API in the browser.

7.2 What Is and Is Not Stored On-Chain

Student name	**Yes — via register()**
Matric number	**Yes — via register()**
Institution	**Yes — via register()**
SHA-256 hash of payload	**Yes — via storeRecord()**
Human-readable label (CGPA summary)	**Yes — via storeRecord()**
Course codes / titles / scores	**No — local only**
Raw CGPA value	**No — in payload label only as string**
Date of birth / PII beyond above	**No — never collected**

Table 7 — On-chain data inventory

---

8 Verification Protocol

Verification is a read-only, gas-free operation callable from any Botchain RPC node. The verifier needs two inputs: the student's wallet address and the SHA-256 hash of the payload the student presents.

- Student shares: their wallet address, the raw JSON payload, and (optionally) the tx hash.
- Verifier independently computes: SHA-256(JSON.stringify(payload, null, 2)).
- Verifier calls: GPAVault.verifyRecord(studentAddress, computedHash).
- Contract returns: (found: bool, timestamp: uint256).
- If found = true, the record was anchored on-chain at the returned timestamp.

Replay resistance: Because the payload includes the wallet address and a generated timestamp, two identical GPA records from different students produce different hashes. A student cannot copy another student's hash and claim it as their own.

---

9 Botchain Integration

9.1 Network Parameters

Parameter	Value
Chain Name	Botchain
Chain ID	`677`
Native Currency	BOT
Decimals	`18`
Consensus	EVM-compatible (PoA / PoS — confirm with Botchain docs)
Solidity Target	`^0.8.20`
Deployment Tool	Foundry (forge create) or Hardhat

Table 8 — Botchain network parameters

9.2 Gas & Cost Model

GPA Vault requires exactly two write transactions per student lifetime (one registration, one or more record stores). Read operations (isRegistered, getStudent, verifyRecord) are free. The contract avoids loops during writes, keeping gas costs predictable:

Operation	Estimated Gas	Notes
register()	~80,000 – 120,000	Depends on string lengths. One-time.
storeRecord()	~60,000 – 90,000	Appends to dynamic array. Per snapshot.
verifyRecord()	0 (view)	Free. Iterates records in reverse.
getStudent()	0 (view)	Free. Direct mapping read.

Table 9 — Gas cost estimates

---

10 Security Considerations

No admin key

The contract has no owner, no pause function, and no upgrade mechanism. Deployed code is immutable. This trades upgradeability for trustlessness.

Pre-image resistance

SHA-256 is pre-image resistant: given a hash, it is computationally infeasible to reverse-engineer the payload. The hash alone reveals nothing about the student's scores.

Sybil resistance

There is no on-chain mechanism preventing a student from registering multiple addresses. This is by design — identity uniqueness is out of scope and should be handled at the application layer (e.g. binding to a government-issued ID off-chain).

String storage limits

Solidity stores dynamic strings on the heap. Very long names or matric numbers will cost more gas but will not revert. Input validation is enforced at the UI layer.

Replay of old hashes

A student could store a previous semester's hash and present it as current. The timestamp field in the payload and the on-chain timestamp together allow verifiers to assess recency. Verifiers should request a fresh hash for time-sensitive decisions.

Browser storage

Semester data is stored in localStorage. Users should be aware that clearing browser data removes their local transcript. Exporting a JSON backup is recommended.

---

11 Roadmap

Phase	Milestone	Status
v1.0	Core contract + GPA calculator + registration flow	Complete
v1.1	JSON export / import of full transcript	Planned
v1.2	Institution verification mode (institution signs student hashes)	Planned
v2.0	Zero-knowledge proof of CGPA range without revealing score (zkSNARK)	Research
v2.1	Multi-chain deployment (Celo, Base, Arbitrum)	Research
v3.0	Decentralised credential marketplace — employers query directly	Vision

Table 10 — Development roadmap

---

12 Conclusion

GPA Vault demonstrates that meaningful academic credential verification does not require centralised databases, institutional buy-in, or complex identity infrastructure. A 120-line Solidity contract, a browser-based calculator, and a SHA-256 hash are sufficient to give any student in the world a tamper-proof, self-sovereign academic record — anchored to a public ledger that will outlast any single institution.

The design prioritises simplicity over features. Future versions will explore zero-knowledge proofs to allow students to prove ranges ("my CGPA is above 4.5") without revealing individual course scores, and institution-cosigned records for use cases where third-party attestation is required.

GPA Vault is open source under the MIT License. Contributions, forks, and institutional integrations are welcome.

---

13 References

[1] NIST FIPS 180-4, Secure Hash Standard (SHS), August 2015.

[2] Wood, G. "Ethereum Yellow Paper: a formal specification of Ethereum, a programmable blockchain." 2014.

[3] Ethereum Improvement Proposal 1193 — Ethereum Provider JavaScript API. https://eips.ethereum.org/EIPS/eip-1193

[4] Solidity Language Documentation v0.8.20. https://docs.soliditylang.org/en/v0.8.20/

[5] ethers.js v5.7 Documentation. https://docs.ethers.org/v5/

[6] Botchain Technical Documentation. https://docs.botchain.example (replace with official URL).

[7] National Universities Commission (NUC) — Grading System Guidelines, Federal Republic of Nigeria.

[8] Web Crypto API — W3C Recommendation. https://www.w3.org/TR/WebCryptoAPI/

GPA Vault is provided as-is under the MIT License. This whitepaper describes the protocol as implemented in v1.0. Smart contracts carry inherent risks; audit before deploying to mainnet.